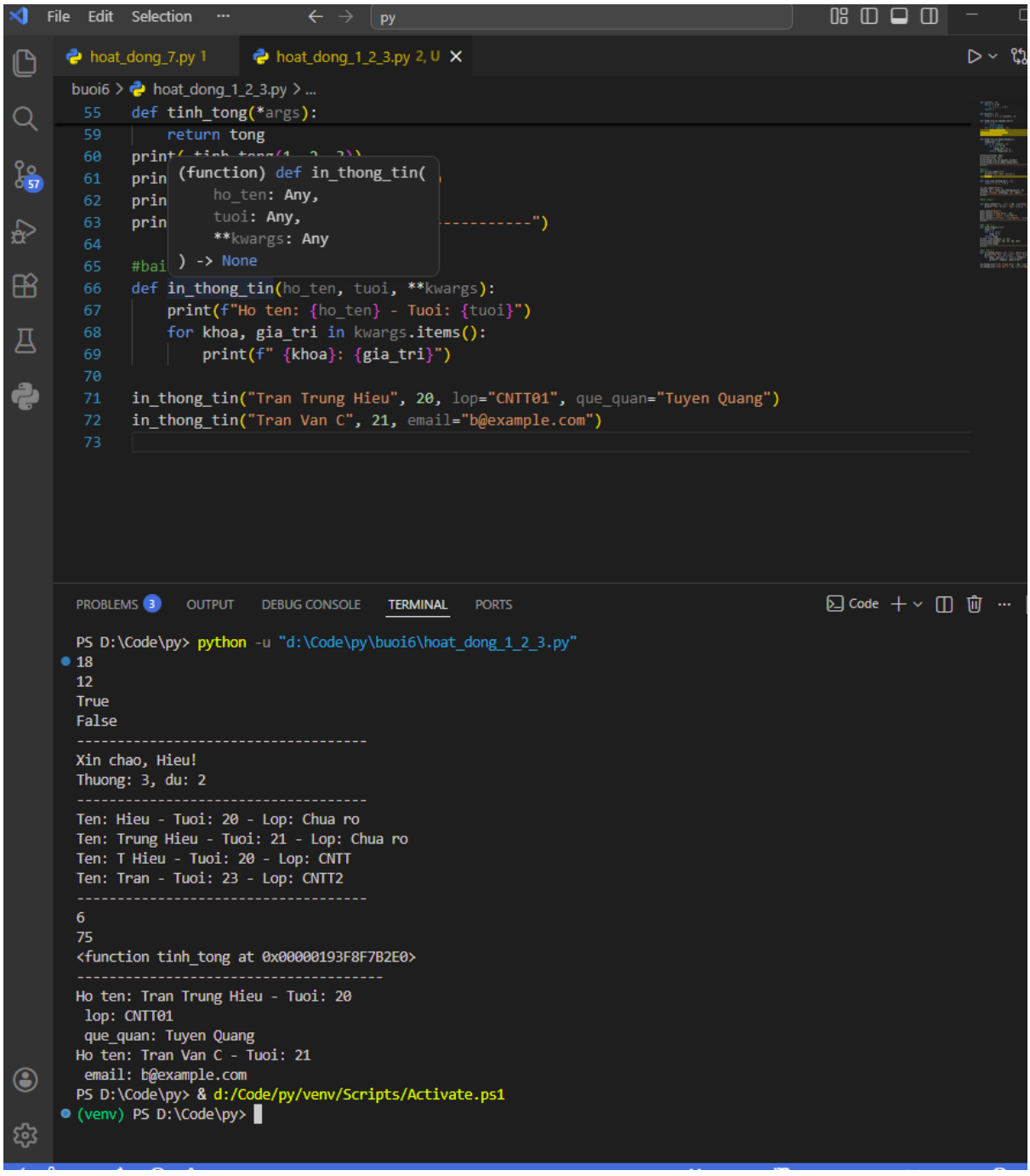
Github: [laptrinhpy/buoi6 at main · trunghieu240606/laptrinhpy]

Hoạt động 1, 2, 3:

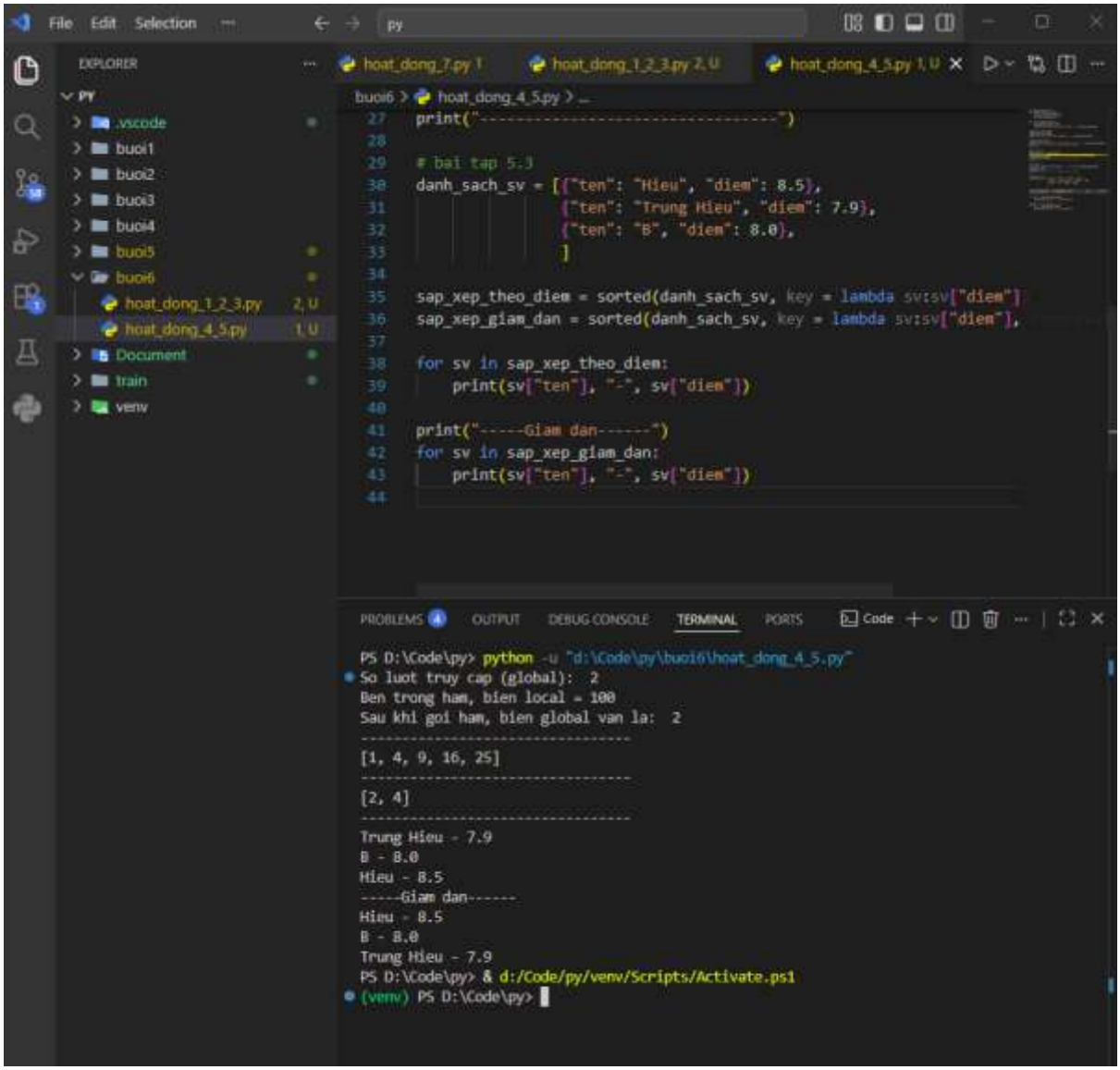


Câu hỏi hoạt động 2:

Khi gọi hàm bằng tham số từ khóa thứ tự truyền vào không quan trọng vì:

- Gán theo tên, không gán theo vị trí: python sẽ đối chiếu và gán giá trị trực tiếp dựa trên tên của tham số đã định nghĩa trong hàm thay vì dựa vào vị trí đứng trước hay sau
- Tự động ánh xạ: Trình thông dịch tự động tìm đúng biến có tên tương ứng để đưa dữ liệu vào, nên đặt biến nào trước hay sau thì hàm vẫn nhận đúng giá trị của biến đó

Hoạt động 4_5:



Câu hỏi hoạt động 4:

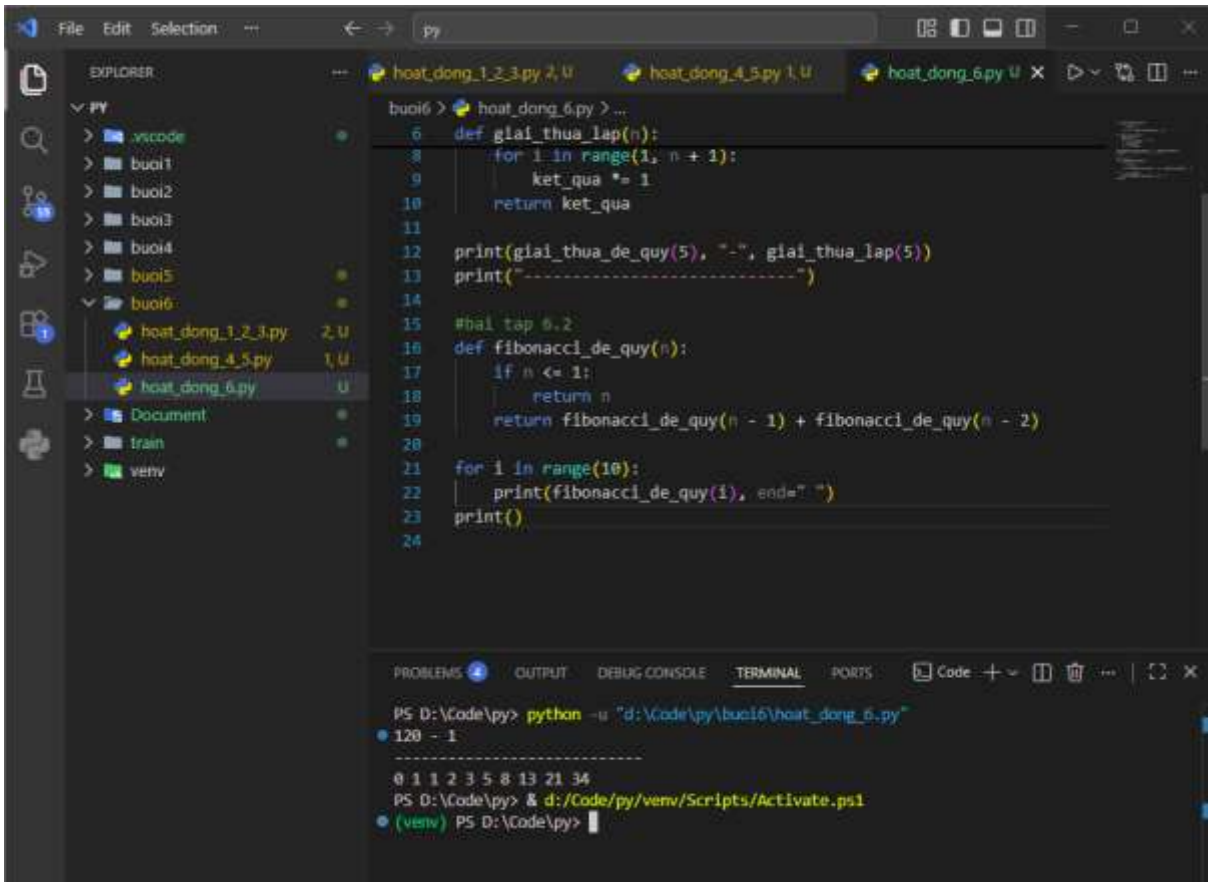
- Trong hàm có phép gán tính toán (+= 1), nên python mặc định hiểu đây là biến riêng (nội bộ) của hàm, chứ không dùng biến ở ngoài
- Để tính được phép cộng, hàm lại phải lấy giá trị cũ ra dùng trước. Vì biến riêng này chưa có giá trị ban đầu nên python không biết lấy gì để cộng, dẫn đến lỗi chưa có giá trị mà đã sử dụng

Câu hỏi hoạt động 5:

Dùng key=lambda linh hoạt hơn vì các lý do sau:

- Không cần sửa đổi cấu trúc dữ liệu:
    - Cách cũ phải cố tình xếp trường cần so sánh lên đầu để máy mặc định so sánh trường đầu tiên trước
    - Dùng lambda giữ nguyên dữ liệu gốc , chỉ cần chỉ định muốn sắp xếp theo tiêu chí nào
- Dễ dàng đổi tiêu chí bất cứ lúc nào: Muốn xếp theo điểm thì trỏ vào điểm, muốn xếp theo tên thì trỏ vào tên, không phải tạo thêm danh sách phụ hay hoán đổi vị trí các phần tử
- Xử lý được các điều kiện phức tạp: lambda cho phép tùy biến linh hoạt hơn, ví dụ: sắp xếp theo độ dài của tên, trị tuyệt đối, hoặc sắp xếp kết hợp nhiều tiêu chí cùng lúc mà không bị gò bó bởi thứ tự ban đầu của dữ liệu

Hoạt động 6:



Câu hỏi hoạt động 6.1

So sánh giữa đệ quy và vòng lặp khi tính giai thừa:

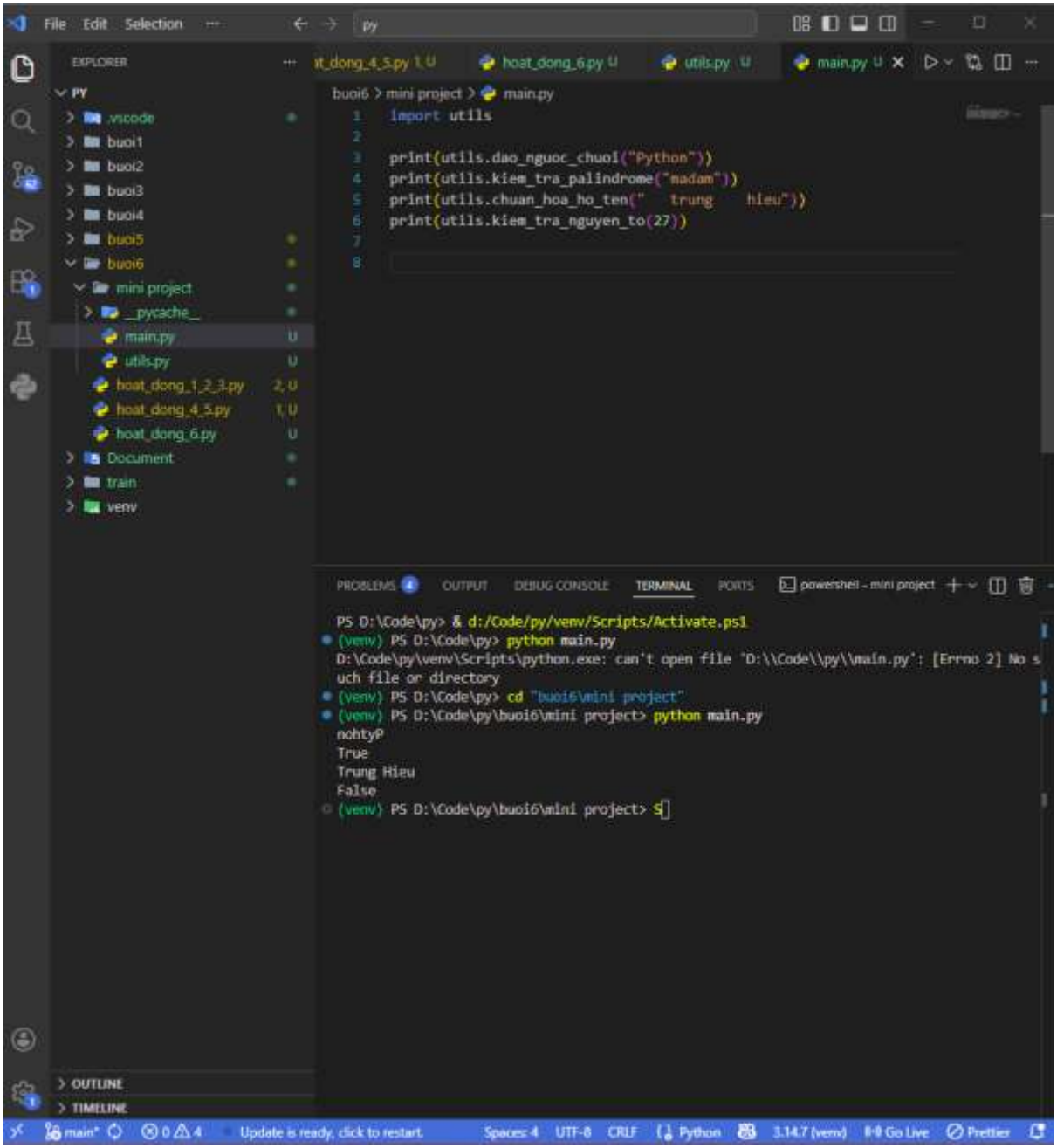
- Bộ nhớ:
    - Vòng lặp for tối ưu hơn vì chỉ dùng một biến lưu kết quả duy nhất trên bộ nhớ
    - Đệ quy tốn bộ nhớ hơn do mỗi lần gọi lại hàm phải tạo thêm một tầng ngăn xếp. Nếu n quá lớn sẽ dễ gây lỗi tràn ngăn xếp
- Tốc độ thực thi vòng lặp chạy nhanh hơn do không mất chi phí đóng/mở hàm liên tục như đệ quy
- Độ trực quan và diễn đạt mã:
    - Đệ quy: Ngắn gọn, bám sát đúng định nghĩa công thức toán học
    - Vòng lặp: Dễ kiểm soát tiến trình chạy từng bước, an toàn và thực tế hơn trong ứng dụng thực tế

Câu hỏi hoạt động 6.2:

Đệ quy Fibonacci tốn kém hơn giai thừa rất nhiều vì cấu trúc phân nhánh và tính toán lặp lại:

- Cây gọi hàm bùng nổ theo cấp số nhân O2^n:
    - Khi tính giai thừa, mỗi lần gọi hàm chỉ kích hoạt 1 lời gọi đệ quy tiếp theo n-1, tạo thành một chuỗi thẳng
    - Với Fibonacci, mỗi bước lại phân nhánh thành 2 lời gọi đệ quy mới (n - 1 và n - 2), khiến số lượng lời gọi hàm tăng theo cấp số nhân
- Trùng lặp tính toán khổng lồ: Chương trình phải liên tục tính đi tính lại các giá trị con giống hệt nhau

Hoạt động 7:



Câu hỏi hoạt động 7:

Hai file cần đặt trong cùng một thư mục để lệnh import utils hoạt động đúng vì cơ chế tìm kiếm module mặc định của python:

- Ưu tiên tìm kiếm thư mục hiện tại: Khi chạy lệnh python main.py, python sẽ tự động đưa thư mục chứa file main.py lên vị trí đầu tiên trong danh sách đường dẫn tìm kiếm module
- Tránh lỗi không tìm thấy module: Khi gặp lệnh import utils, python sẽ lập tức tìm file utils.py ngay trong thư mục đó. Nếu đặt ở thư mục khác, python sẽ không tìm thấy và báo lỗi